

Desafio 3 — O melhor treino para o seu problema

Otimizadores e agendas de taxa de aprendizado, observados por dentro

Prof. Marcelo Keese Albertini • GBC073 — Inteligência Computacional • FACOM/UFU

2026

Em uma frase

Cada grupo encontra, **no problema do seu projeto**, a melhor combinação de *otimizador* e *agenda de taxa de aprendizado*, e mostra *por que* ela é a melhor usando três sinais do treino: **a evolução da perda, a norma do gradiente e a norma dos parâmetros**.

1 O que registrar

A cada k passos ($10 \leq k \leq 50$), *globalmente* e *por camada* (no mínimo: primeira, uma do meio e última):

Sinal	Definição	O que revela
Perda	$\mathcal{L}_t^{\text{tr}}$ por lote; $\mathcal{L}^{\text{val}}$ periódica	convergência, platôs, picos, sobreajuste
Norma do gradiente	$\ \nabla_{\theta} \mathcal{L}_t\ _2$, antes do <i>clipping</i>	estabilidade: picos = passo grande demais; ≈ 0 cedo = gradiente sumindo
Norma dos parâmetros	$\ \theta_t\ _2$	efeito líquido de η , <i>weight decay</i> e normalização
Razão de atualização	$\ \theta_{t+1} - \theta_t\ / \ \theta_t\ $	quanto cada camada muda por passo ($\sim 10^{-3}$ é uma boa referência)
Taxa efetiva	η_t	confirma que a agenda fez o que se esperava

```
# depois de loss.backward(), antes do clipping e de opt.step():
g = torch.sqrt(sum((p.grad**2).sum() for p in model.parameters() if p.grad is not None))
th = torch.sqrt(sum((p**2).sum() for p in model.parameters()))
antes = {n: p.detach().clone() for n, p in model.named_parameters() if n in CAMADAS}
opt.step(); sched.step()
rho = {(p - antes[n]).norm() / antes[n].norm().item()
        for n, p in model.named_parameters() if n in CAMADAS}
log.append(dict(passo=t, perda=loss.item(), grad=g.item(), theta=th.item(),
               lr=opt.param_groups[0]["lr"], **{f"rho/{n}": v for n, v in rho.items()}))
```

Dica: `clip_grad_norm` devolve a norma total antes do corte. Use o mesmo registro em **todas** as execuções.

2 O que comparar

Otimizadores (mínimo 4): SGD com momento (com e sem Nesterov), Adam, AdamW e, **obrigatoriamente, um otimizador da família Muon**. Um quinto à escolha é bem-vindo (RMSprop, NAdam, Lion, Schedule-Free...). **Agendas (mínimo 4):** constante (controle), cosseno, *warmup* + cosseno e uma à escolha (*one-cycle*, degraus, WSD, ReduceLR0nPlateau...). Ver `torch.optim` e `lr_scheduler`.

Muon (obrigatório). Para cada matriz de pesos W , o Muon acumula o momento M e, em vez de usar M como passo, usa sua versão *ortogonalizada*: se $M = U\Sigma V^T$, o passo é $\approx UV^T$, calculado com 5 iterações de Newton–Schulz. Todas as direções singulares recebem o mesmo tamanho de passo, inclusive as raras, que o SGD e o Adam quase ignoram. Ele vale **só para matrizes 2-D das camadas**

ocultas; vieses, ganhos de normalização, *embeddings* e a camada de saída continuam com AdamW. Kernels de convolução podem entrar como matriz achatada ($C_{out}, C_{in}k_hk_w$) em implementação própria; com `torch.optim.Muon` ficam no AdamW. Vale `torch.optim.Muon` (PyTorch ≥ 2.9 ; apenas 2-D), uma implementação própria ou uma variante (NorMuon, Muon com ajuste de RMS do Moonshot...), desde que a ortogonalização esteja lá.

```
muon_p = [p for n, p in model.named_parameters() if p.ndim == 2 and "head" not in n and "emb" not in n]
ids = {id(p) for p in muon_p}; adamw_p = [p for p in model.parameters() if id(p) not in ids]
opts = [torch.optim.Muon(muon_p, lr=0.02, momentum=0.95, weight_decay=0.01),
        torch.optim.AdamW(adamw_p, lr=3e-4, weight_decay=0.01)]
scheds = [agenda(o) for o in opts] # a MESMA agenda nos dois otimizadores
# no laço: for o in opts: o.step(); for s in scheds: s.step()

def ortogonaliza(G, passos=5): # Newton-Schulz, para quem implementar o próprio
    a, b, c = 3.4445, -4.7750, 2.0315
    X = G / (G.norm() + 1e-7); T = X.size(0) > X.size(1)
    if T: X = X.T
    for _ in range(passos):
        A = X @ X.T; X = a * X + (b * A + c * A @ A) @ X
    return X.T if T else X
```

No registro, a razão de atualização por camada é onde o Muon se diferencia: compare-a com a do AdamW.

3 Protocolo

1. **Baseline:** a receita atual do grupo, 3 sementes.
2. **Varredura de η :** para cada otimizador, ≥ 5 valores em grade logarítmica, agenda *warmup* + cosseno, 1 semente. Se o melhor cair na borda, estenda a grade. No Muon, varra o η das matrizes (tipicamente 10^{-3} a 10^{-1}) com a parte AdamW no melhor η já encontrado para o AdamW.
3. **Agendas:** para os 2 melhores otimizadores **e o Muon** (se não estiver entre eles), com o melhor η de cada, as 4 agendas, 1 semente.
4. **Confirmação:** as 3 melhores combinações com 3 sementes. Só há vencedor se a diferença superar a variação entre sementes.

Regras

Mesmo orçamento (passos e tamanho de lote) para todas as comparações • seleção na validação, teste usado uma vez • todas as execuções reportadas, inclusive as que divergiram • comparar otimizadores com η não ajustada invalida a comparação • orçamento reduzido nas etapas 2–3 é permitido se for igual para todos e declarado • sem um otimizador da família Muon, o critério de protocolo vale zero.

4 Como ler os sinais

Sintoma	Diagnóstico provável	O que testar
picos em $\ g\ $ seguidos de saltos na perda	passo grande demais	η menor, <i>clipping</i>
picos só no início	estatísticas do Adam / inicialização	<i>warmup</i>
platô com razão de atualização $\lesssim 10^{-5}$	passo pequeno demais	η maior
platô com razão de atualização $\gtrsim 10^{-1}$	oscilação no vale	η ou momento menor
$\ \theta\ $ cresce sem estabilizar	falta regularização	AdamW, <i>weight decay</i>
$\ g\ $ de uma camada $\ll$ das demais	camada não aprende	inicialização, normalização
treino cai, validação sobe	sobreajuste, não instabilidade	regularização, parada antecipada
queda brusca no fim do cosseno/WSD	decaimento final "assenta" no vale	esperado

Alinhe perda, $\|g\|$, $\|\theta\|$ e η_t no mesmo eixo de passos: quase todo diagnóstico sai de ler a mesma coluna vertical nos quatro gráficos.

5 Entrega, avaliação e prazo

O que entregar

Pasta desafio3/ no repositório do grupo: `treino.py` configurável (otimizador, η , agenda, semente) • `experimentos.csv` (uma linha por execução: configuração, métrica de validação, divergiu?, tempo) • `logs/` de todas as execuções • `painel.pdf`: os quatro sinais alinhados para a melhor configuração, a baseline, a melhor com Muon e uma instável, e a métrica $\times \eta$ por otimizador • `slide.pdf`: **um slide**, 3 minutos — receita vencedora, ganho $\pm$ desvio, e um diagnóstico lido nos sinais que a justifica.

Critério	Pontos
Protocolo justo (η ajustada por otimizador, orçamento fixo, tudo reportado)	3
Interpretação sustentada pelos sinais, não só pela métrica final	3
Instrumentação completa e painel legível	2
Ganho sobre a baseline maior que a variação entre sementes (ou explicação de por que não há)	1
Reprodutibilidade de qualquer linha de <code>experimentos.csv</code>	1

Entrega: quarta-feira, 14 de outubro de 2026, até 12h00 (meio-dia). Vale o último *commit* enviado ao repositório até esse horário; depois dele, nada é considerado. Ritmo sugerido: baseline com registro ligado até sexta (9/10); varreduras de η e de agendas no fim de semana; confirmação com 3 sementes, painel e slide até terça (13/10).

Leituras: Goyal et al., *Accurate, Large Minibatch SGD* (2017) • Loshchilov & Hutter, *SGDR* (2017) e *Decoupled Weight Decay* (2019) • Smith, *Cyclical Learning Rates* (2017) • Wilson et al., *The Marginal Value of Adaptive Gradient Methods* (2017) • Schmidt et al., *Descending through a Crowded Valley* (2021) • Jordan, *Muon: An optimizer for hidden layers* (2024) • Liu et al., *Muon is Scalable for LLM Training* (2025) • Hägele et al., *Scaling Laws and Compute-Optimal Training Beyond Fixed Training Durations* (2024).